PROYECTO UT03-UT04: Servicio Web REST con FTP y Email

IES PALOMERAS VALLECAS

IGNACIO VILLACIS

Contenido

1. INTRODUCCIÓN Y OBJETIVOS 2
2. ESTADO DEL SERVIDOR VSFTPD (DOCKER) 2
Comando de Ejecución 2
Comprobación de estado 2
3. ESTADO DEL SERVICIO REST (SPRING BOOT) 3
Estructura del Proyecto 4
4. COLECCIÓN POSTMAN Y PRUEBAS 4
4.1. Subir Archivo (POST /upload-file) 4
4.2. Descargar Archivo (POST /download-file) 5
4.3. Eliminar Archivo (DELETE /delete-file) 5
4.4. Enviar Correo (POST /send-email) 6
5.1. El Controlador REST (ApiController.java) 6
Puntos clave del código: 7
5.2. Servicio FTP (FtpService.java) 7
5.3. Servicio de Email (EmailService.java) 8
6. DIFICULTADES ENCONTRADAS Y SOLUCIÓN 9
7. REFERENCIAS 9

1. INTRODUCCIÓN Y OBJETIVOS

En este proyecto, tal como solicitó el cliente ficticio de la práctica, hemos desarrollado una solución backend completa que integra varios protocolos de comunicación. El objetivo principal era crear un sistema desacoplado donde un servicio REST (construido con Spring Boot) actúa como intermediario para gestionar archivos en un servidor FTP y enviar notificaciones por correo electrónico.

La necesidad surge de tener un entorno de trabajo centralizado para empleados, donde no accedan directamente al FTP, sino a través de una API controlada y segura.

2. ESTADO DEL SERVIDOR VSFTPD (DOCKER)

Para el servidor FTP, no instalamos nada directamente en la máquina host ("mi pc"), sino que optamos por la virtualización ligera con **Docker**. Esto nos permite tener el entorno aislado y limpio.

Utilizamos la imagen fauria/vsftpd que es un estándar robusto para este tipo de prácticas.

Comando de Ejecución

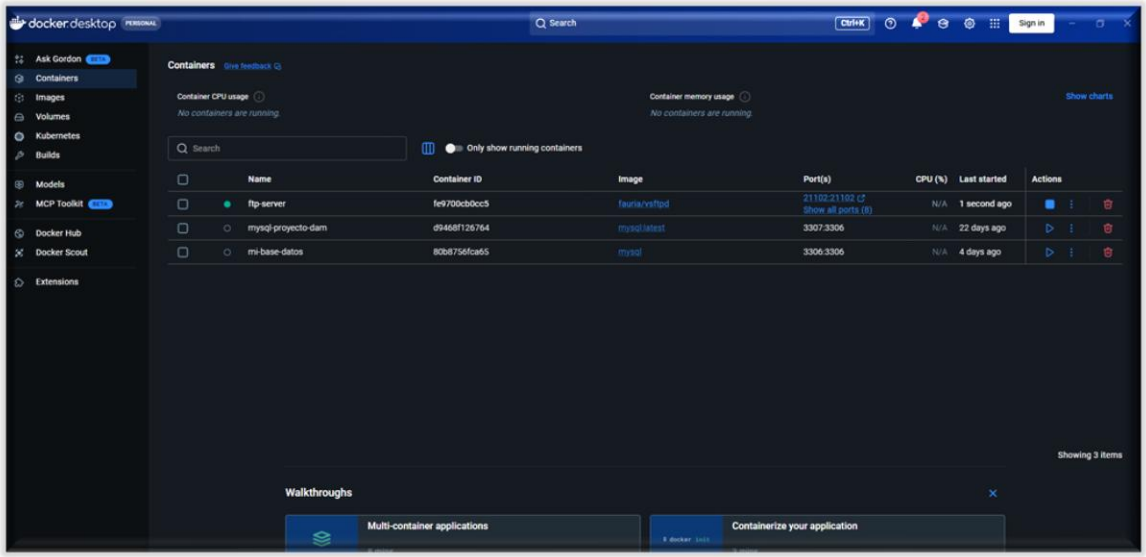
Para levantar el contenedor, utilicé el siguiente comando en la terminal, asegurando el mapeo de puertos y volúmenes para que los archivos no se pierdan al apagar el contenedor:

```
docker run --platform linux/amd64 -d -v
/Users/[TU_USUARIO]/Documents/ftp:/home/vsftpd \

-p 21:21 -p 21100-21110:21100-21110 \

-e FTP_USER=myuser -e FTP_PASS=mypass \

-e PASV_ADDRESS=127.0.0.1 -e PASV_MIN_PORT=21100 -e
PASV_MAX_PORT=21110 \

--name ftp-server fauria/vsftpd
```

Nota: Tuve especial cuidado con el rango de puertos pasivos (21100-21110) porque es donde suele fallar la conexión de datos.

Comprobación de estado

A continuación, se muestra que el contenedor está corriendo correctamente (Up) y escuchando en el puerto 21.

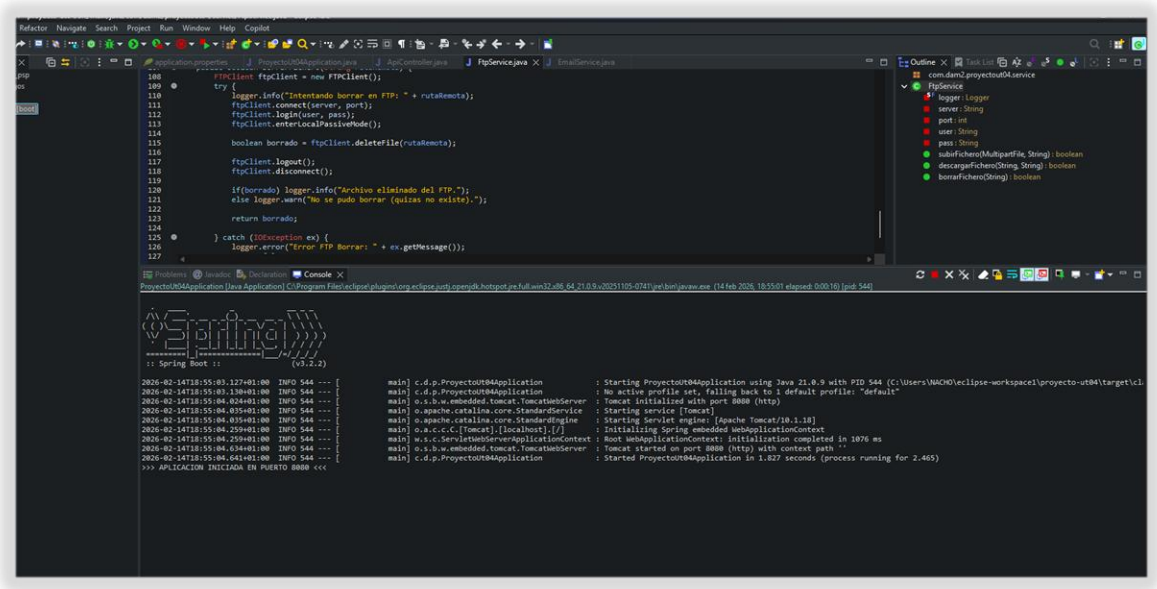


También verifiqué la conexión inicial usando un cliente FTP básico con el explorador de archivos antes de meter código Java, para descartar fallos de red.

3. ESTADO DEL SERVICIO REST (SPRING BOOT)

El corazón del proyecto es la aplicación Spring Boot ProyectoUt04Application. Se ha configurado para correr en el puerto 8080.

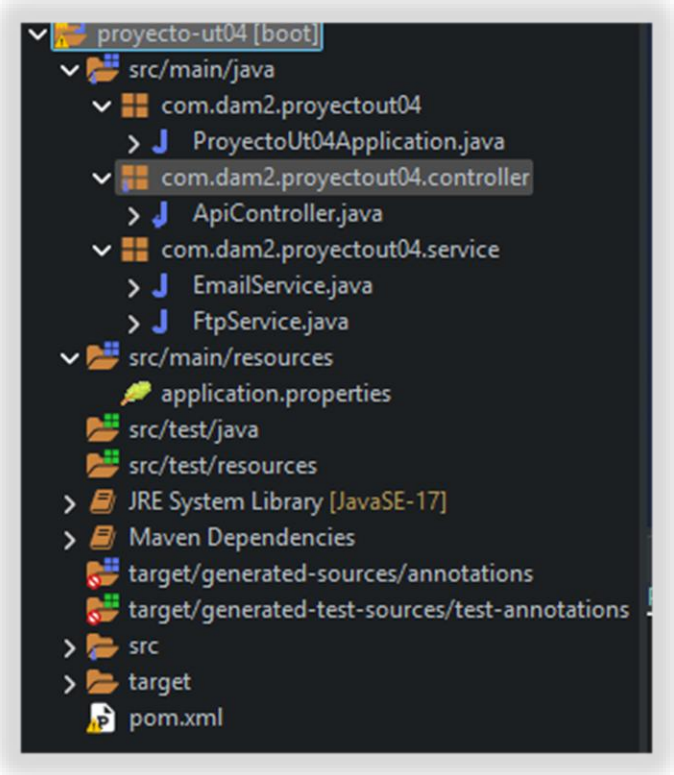
Al iniciar la aplicación en el IDE Eclipse, podemos ver en la consola los logs personalizados que hemos programado con LoggerFactory para tener trazabilidad de lo que ocurre.



Estructura del Proyecto

El proyecto sigue una arquitectura en capas clásica y limpia:

1. **Controller:** Recibe las peticiones HTTP.
2. **Service:** Contiene la lógica dura (conectar al FTP, mandar el mail).
3. **App:** Clase principal.



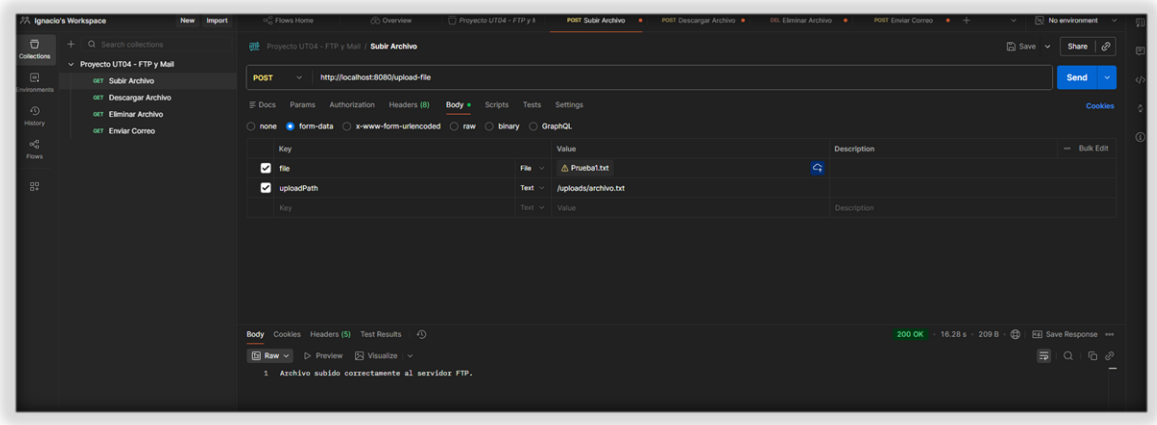
4. COLECCIÓN POSTMAN Y PRUEBAS

Se ha creado una colección en Postman llamada **"Proyecto UT04 FTP Mail"** que incluye los 4 endpoints solicitados. A continuación, muestro la evidencia de que todos devuelven un **Status 200 OK**.

4.1. Subir Archivo (POST /upload-file)

Este endpoint recibe un archivo MultipartFile y lo envía al servidor FTP.

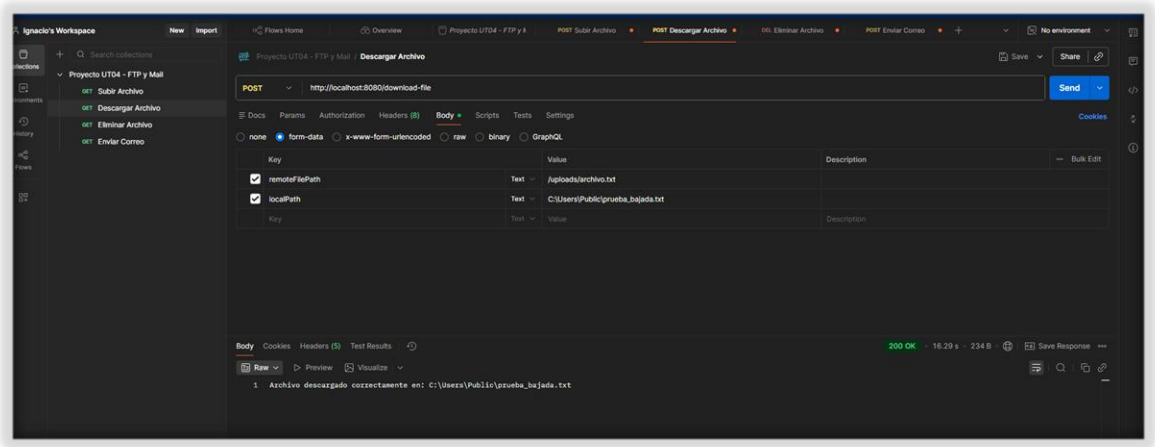
- **Body:** form-data (key: file).



4.2. Descargar Archivo (POST /download-file)

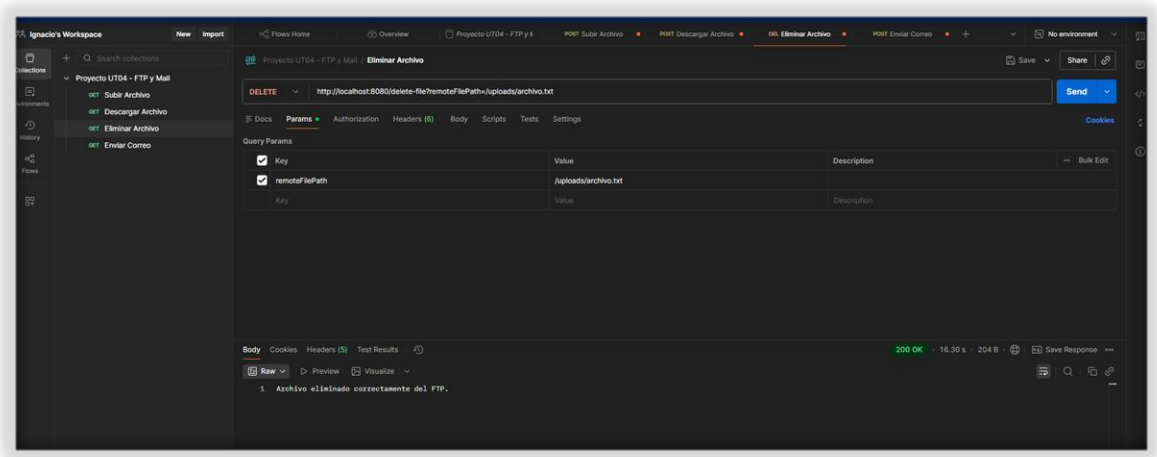
Descarga un archivo desde el FTP y lo guarda en una ruta local de mi ordenador.

- **Params:** remoteFilePath (ruta en el FTP) y localPath (donde lo quiero guardar).



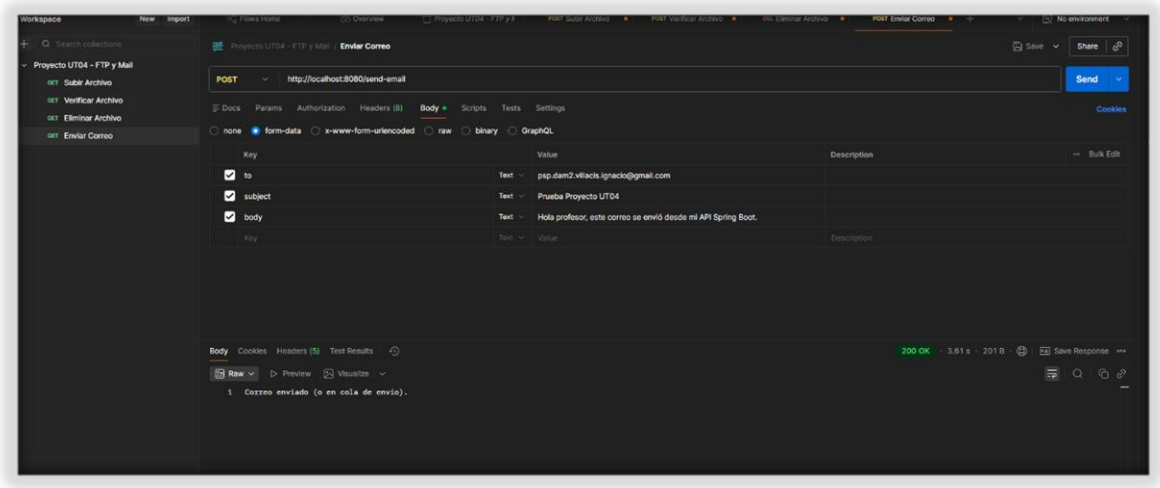
4.3. Eliminar Archivo (DELETE /delete-file)

Borra el archivo del servidor remoto. Es útil para limpiar el servidor después de las pruebas.

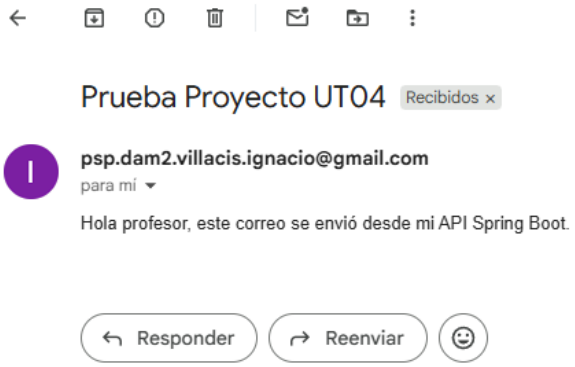


4.4. Enviar Correo (POST /send-email)

Este fue el endpoint extra. Se conecta al servidor SMTP de Google (configurado en application.properties) y envía un correo de texto plano.



Evidencia de recepción: Aquí muestro cómo llegó el correo a la bandeja de entrada real:



5. DESARROLLO E IMPLEMENTACIÓN

Para que el proyecto sea modular y fácil de mantener, he seguido una arquitectura en capas, separando la lógica de negocio (Servicios) de la gestión de peticiones HTTP (Controlador). A continuación, explico en detalle las clases más importantes que he programado en Eclipse.

5.1. El Controlador REST (ApiController.java)

Esta clase es el "cerebro" de la API. Se encarga de recibir las peticiones de Postman y delegar el trabajo a los servicios correspondientes.

Puntos clave del código:

- @RestController: Con esta anotación, le digo a Spring que esta clase va a manejar peticiones web y que las respuestas no son páginas HTML, sino datos directos (texto o JSON).

Inyección de Dependencias (@Autowired): En lugar de crear objetos con new FtpService(), dejo que Spring "inyecte" las instancias automáticamente. Esto hace que el código sea más limpio y eficiente.

- Endpoints: Definí los métodos con @PostMapping y @DeleteMapping para mapear las URLs exactas que pedía el PDF (/upload-file, /download-file, etc.).
- Logging: Uso Logger para ir dejando trazas en la consola cada vez que llega una petición, lo cual es vital para saber qué está pasando en tiempo real.

```java
@RestController
public class ApiController {


    private static final Logger logger = LoggerFactory.getLogger(ApiController.class);

    @Autowired
    private FtpService ftpService;

    @Autowired
    private EmailService emailService;


    @PostMapping("/upload-file")
    public ResponseEntity<String> uploadFile(
            @RequestParam("file") MultipartFile file,
            @RequestParam(value = "uploadPath", required = false) String uploadPath) {

        logger.info(">>> API: Recibida petición para subir archivo: " + file.getOriginalFilename());


        String destino = (uploadPath != null && !uploadPath.isEmpty()) ? uploadPath : file.getOriginalFilename();

        boolean exito = ftpService.subirFichero(file, destino);

        if (exito) {
            return ResponseEntity.ok("Archivo subido correctamente al servidor FTP.");
        } else {
            return ResponseEntity.status(500).body("Error al subir el archivo.");
        }
    }
```

5.2. Servicio FTP (FtpService.java)

Aquí implementé toda la lógica de comunicación con el contenedor Docker. Utilicé la librería Apache Commons Net, que es la estándar para estas tareas en Java.

Detalle técnico de la implementación:

1. Conexión y Autenticación: Utilizo la clase FTPClient. Leo el usuario y contraseña desde el archivo application.properties usando la anotación @Value, para no tener credenciales escritas directamente en el código Java (hardcoded).

2. Modo Pasivo (enterLocalPassiveMode): Esta línea es crítica. Al trabajar con Docker, si no activamos el modo pasivo, el firewall o la red de Docker bloquean la transferencia de datos.

3. Transferencia Binaria: Configuré ftpClient.setFileType(FTP.BINARY_FILE_TYPE). Si no hacía esto, los archivos de texto subían bien, pero las imágenes o PDFs llegaban corruptos.

4. Manejo de Streams: Para la descarga, no guardo el archivo en memoria RAM (lo que podría colapsar el servidor con archivos grandes), sino que uso un BufferedOutputStream para escribirlo directamente en el disco duro del cliente byte a byte.

```java
// --- METODO 1: SUBIR FICHERO (POST /upload-file) ---
public boolean subirFichero(MultipartFile file, String rutaDestino) {
    FTPClient ftpClient = new FTPClient();
    try {
        logger.info("Conectando a FTP para subir: " + file.getOriginalFilename());
        ftpClient.connect(server, port);
        ftpClient.login(user, pass);
        ftpClient.enterLocalPassiveMode(); // Vital para Docker
        ftpClient.setFileType(FTP.BINARY_FILE_TYPE); // Para que no se rompan PDFs o imagenes

        // Si el PDF manda una ruta especifica (uploadPath), intentamos usarla
        // Si no, usamos el nombre original en la raiz
        String nombreRemoto = (rutaDestino != null && !rutaDestino.isEmpty()) ? rutaDestino : file.getOriginalFilename();

        InputStream inputStream = file.getInputStream();
        boolean hecho = ftpClient.storeFile(nombreRemoto, inputStream);
        inputStream.close();

        ftpClient.logout();
        ftpClient.disconnect();

        if(hecho) logger.info("Subida correcta a: " + nombreRemoto);
        else logger.error("Fallo al subir archivo FTP");

        return hecho;

    } catch (IOException ex) {
        logger.error("Error FTP Subir: " + ex.getMessage());
        return false;
    }
}
```

5.3. Servicio de Email (EmailService.java)

Para el envío de correos, aproveché la integración de Spring Boot Starter Mail, que simplifica muchísimo el uso de la librería JavaMail nativa.

Cómo funciona:

- Spring Boot configura automáticamente el objeto JavaMailSender usando los datos que puse en application.properties (host de Gmail, puerto 587, usuario y contraseña de aplicación).
- En el código, solo tengo que crear un objeto SimpleMailMessage, definir el remitente (setFrom), el destinatario (setTo) y el mensaje, y darle a send.
- He incluido un bloque try-catch para capturar cualquier error de red (como falta de internet) y que el servidor no se caiga, simplemente registrando el error en el log.

```java
@Service
public class EmailService {

    private static final Logger logger = LoggerFactory.getLogger(EmailService.class);

    @Autowired
    private JavaMailSender mailSender;


    @Value("${spring.mail.username}")
    private String remitente;

    public void enviarCorreo(String destinatario, String asunto, String cuerpo) {
        try {
            logger.info("Enviando correo a: " + destinatario);

            SimpleMailMessage mensaje = new SimpleMailMessage();
            mensaje.setFrom(remitente);
            mensaje.setTo(destinatario);
            mensaje.setSubject(asunto);
            mensaje.setText(cuerpo);

            mailSender.send(mensaje);

            logger.info("Correo enviado con EXITO.");
        } catch (Exception e) {
            logger.error("FALLO al enviar correo: " + e.getMessage());
        }
    }
}
```

6. DIFICULTADES ENCONTRADAS Y SOLUCIÓN

El principal problema que tuve fue un Error 500 al subir archivos. Aunque la conexión era correcta, el servidor FTP no dejaba escribir. Investigando, me di cuenta de que Docker intentaba guardar el archivo en la carpeta del usuario (/home/vsftpd/myuser), pero en mi Windows yo solo había creado la carpeta raíz ftp. Solución: Creé manualmente la carpeta myuser dentro de C:/Users/Public/ftp y reinicié el contenedor.

También tuve que activar la Verificación en dos pasos de Google para generar una contraseña de aplicación, ya que la contraseña normal de Gmail no funcionaba con Java.

7. REFERENCIAS

Material del aula virtual: Me he basado principalmente en los ejemplos de código proporcionados por el profesor en la plataforma. He utilizado como guía las clases ClienteFTP1.java y DescargarFichero.java para entender la lógica de conexión y gestión de archivos**.**

Ejemplos de correo: Para la parte del servicio de email, me serví de los archivos EnviarCorreoSencillo.java y EnviarCorreoServerLocal.java para ver cómo configurar el servidor SMTP correctamente.

Tips del enunciado: Seguí las instrucciones del PDF de la práctica para los comandos de Docker y las rutas de los endpoints solicitados.

Documentación de Apache Commons Net: Consulté la documentación oficial de la librería FTPClient para profundizar en el uso de métodos específicos como enterLocalPassiveMode(), que es necesario para que el FTP funcione dentro de Docker.